MIT8321 Forensics and Ethical Hacking (Virtual Lab)

Continuous Assessment, 2026/2027 Academic Session

Lab Exercise 4: Evidence Collection and Preservation

Case SB-2026-001, SecureBank Cyber Fortress lab

Student name: Feyifoluwa Abioye

Student Id: 301836014

Submission date: Nov 1st, 2026

GitHub repository: https://github.com/obiwanpelosi/mit8321-ca-exercise4

Investigation date: 1 November 2026 (all times UTC)

Contents

1. Executive summary....3
2. Introduction....3
3. Scope and ethics....3
4. Lab environment and tools....3
5. Methodology....4
6. Evidence collection....5
7. Evidence register....5
8. Hash verification and chain of custody....6
9. Observations and recommendations....7
10. Limitations....7
11. Conclusion....8
12. Reflection....8
Appendix A: Collection script and lab configuration change....9
Appendix B: Screenshots S01–S12....13
Appendix C: Hash values....24
Appendix D: Collection log....26
Appendix E: Investigation logbook....27

1. Executive summary

This report records how digital evidence was collected from, and preserved for, the SecureBank Cyber Fortress lab under case SB-2026-001 on 1 November 2026. Working only inside the isolated `securebank-lab` virtual machine, the investigator generated controlled login and attack activity, then ran one collection script that captured **24 evidence items (EV-01 to EV-24)** in order of volatility. The script hashed each item with SHA-256 and MD5, wrote an evidence register and chain of custody, and sealed the evidence read-only in a hashed archive (S08, S09).

All 24 SHA-256 hashes verified after sealing and again after transfer to the analysis computer (S10, S11). A tamper test on a working copy changed that copy's hash completely while the original stayed unchanged (S10). The evidence also shows three important weaknesses: an SQL-injection string was accepted as a successful login (EV-09, EV-21); Flask debug mode is on and the debugger PIN is written to the security log (EV-09, EV-10, EV-21); and database passwords and the session secret are hard-coded in plain text (EV-18, EV-20).

2. Introduction

SecureBank Cyber Fortress is a deliberately vulnerable online-banking application used for MIT8321 training. In the course scenario, the security team suspects the application has been targeted through suspicious logins, injection attacks and unusual network activity, and asks an investigator to gather evidence that can be trusted. The lab runs six Docker containers: the Flask banking application, a MySQL database, a Kali Linux attacker container, and an Elasticsearch, Logstash and Kibana (ELK) log stack (EV-05, EV-08).

The brief instructs candidates to attempt only one lab exercise. This report covers **Lab Exercise 4, Evidence Collection and Preservation**, whose objectives are to collect relevant forensic evidence, preserve its integrity, demonstrate chain of custody and apply good documentation practice. Its deliverables are the evidence folder, evidence register, hash values file, chain-of-custody table, screenshots of the collection process and an explanation of the preservation method. They appear in Sections 7 and 8 and Appendices A to E, and the same files are held in the public GitHub repository named on the title page.

3. Scope and ethics

This assessment covered evidence collection and preservation for case SB-2026-001 only. The single system examined was a dedicated Ubuntu VM, `securebank-lab`, running on the investigator's own Mac, and the six SecureBank containers inside it. The VM has one network interface, on Multipass's private 192.168.252.0/24 network shared with the host (EV-04, S04); no bridged adapter was configured.

No real, public, school or third-party system was scanned, tested or accessed. Web requests were sent with curl from inside the VM. The lecturer's attack script ran from the `kali-attacker` container and targeted only the `securebank-app` container (logbook, Appendix E; EV-09). Only test accounts defined in the lab's own `db_init.sql` were used (EV-22). No lab data was deleted or modified, and the collected evidence was never edited after it was hashed (S10, S11).

4. Lab environment and tools

Table 1 lists the systems examined. Container addresses come from the lab network inspection (EV-05) and published ports from the container list (EV-08).

Component	Image or platform	Address	Published on VM
VM securebank-lab	Ubuntu 22.04 (Multipass), kernel 5.15.0-198, aarch64, Docker 29.1.3 (EV-01)	192.168.252.3 (EV-04)	SSH 22 (EV-02)
securebank-app	secure-cyber-fortress-vulnerable-app (Flask)	172.18.0.2	0.0.0.0:8080 → 5000
securebank-mysql	mysql:8.0	172.18.0.4	none (3306 internal)
kali-attacker	secure-cyber-fortress-kali-tools	172.18.0.7	none
elasticsearch	elasticsearch:7.17.9	172.18.0.3	0.0.0.0:9200
logstash	logstash:7.17.9	172.18.0.6	0.0.0.0:5000 tcp/udp
kibana	kibana:7.17.9	172.18.0.5	0.0.0.0:5601

Table 1: Lab systems (EV-01, EV-04, EV-05, EV-08)

Tools: Multipass; Docker and Docker Compose v2; Git; curl; `ss`, `netstat`, `ip`, `ps`, `docker top`, `docker inspect` and `docker logs` for volatile data; `find`, `ls` and `df` for the file system; `sha256sum` and `md5sum` for hashing; `tar` for sealing; `shasum` on macOS for re-verification; and nmap, which ran only inside the lecturer's attack script.

Deviations from the lab guides:

- Multipass was used instead of VirtualBox.
- The image was changed from `mysql:5.7` to `mysql:8.0` because the VM runs on ARM (`aarch64`). This change is recorded in the Git diff in EV-18 and shown in Appendix A.
- The repository's sample evidence from a February 2026 run was moved to `sample_evidence_from_repo/` before the lab started, so every item submitted was generated in this investigation (EV-13, EV-18).
- `setup.sh` was not used; the lab was started directly with Docker Compose v2.
- The course repository's own compose file was used, as the deployment guide directs. It publishes ports on `0.0.0.0` rather than `127.0.0.1` and uses Elastic 7.17.9 rather than 7.17.18 (EV-08, EV-20).
- Smaller procedural changes (login page fetched with curl, no logout route, screenshot handling and the tamper-test working copy) are explained in Sections 6, 8 and 10.

5. Methodology

The method was built around five preservation principles, all implemented in one script, `collect_evidence.sh` (Appendix A), so that the procedure can be repeated and audited.

- **Separation from the live system.** Evidence was written to `~/case_SB-2026-001/evidence/`, outside the lab repository. The lab's own `evidence/` folder is live, because the application keeps appending to `securebank.log`, so nothing collected could be changed by a running process.
- **Order of volatility (RFC 3227).** The most changeable data, such as system time, network sockets and running processes, was captured first. Logs, file-system state and static configuration followed. The eight volatile items were captured within about one second, 14:20:22Z to 14:20:23Z (EV-01 to EV-08).
- **Documented, consistent time.** Every capture begins with a header recording the case ID, collection time in UTC, investigator, host, source and exact command (for example EV-10). The collection log records each command with its UTC time (Appendix D), and the logbook records every manual action (Appendix E). The VM clock was NTP-synchronised (EV-01).
- **Faithful copies.** Existing files were copied with `cp -p` so their original modification times were kept; command output was redirected straight into files.

- **Integrity and sealing.** Each item was hashed with SHA-256 (primary) and MD5 (secondary) into `hashes.txt` and `hashes_md5.txt`. The register was then generated, the evidence folder made read-only with `chmod -R a-w`, and the folder archived to `SB-2026-001_evidence.tar.gz` with its own SHA-256. The hash files and register were also made read-only. The script refuses to run if the evidence folder already holds files, so collected evidence cannot be overwritten by a second run.

Hashes were re-verified after sealing, and again after transfer to the Mac where this report was written. Each check was recorded as a chain-of-custody entry.

6. Evidence collection

First, controlled activity was generated so the application log would hold relevant events. The login page was fetched with curl from inside the VM rather than with a browser on the host, so that every request originated inside the lab; S06 therefore shows the HTTP response, not a browser window. User `jane` then failed to log in three times and succeeded on the fourth attempt, and `john` logged in and opened the accounts page. The application has no logout route, so john's session was ended by discarding its cookie (Appendix E). The lecturer's attack script then ran from the Kali container. `securebank.log` recorded all of this (S07, EV-09).

The collection script then ran from 14:20:22Z to 14:20:26Z and reported 24 items (S08). The results are summarised in Table 2.

Group	Items	What it captures and why it matters
Volatile	EV-01 to EV-08	Clock and NTP status, VM and container sockets, interfaces, container IPs, VM and container processes, container status. Shows what was running and listening at collection time; it is lost on reboot.
Logs	EV-09 to EV-12	Application security log, attack timeline, app console output, and an inventory of log files on the VM. These are the primary record of user and attack events.
File system	EV-13 to EV-17	Lab file permissions and owners, app-container permissions, SUID binaries on the VM and in the app container, disk usage. Shows exposure and privilege risk.
Configuration	EV-18 to EV-24	Lab commit and local changes, `docker inspect` of the app, compose file, source code, database script, Dockerfile, Logstash pipeline. Explains why the behaviour seen in the logs was possible.

Table 2: Evidence groups collected

The SUID review found 14 binaries on the VM's root file system, all standard Ubuntu ones such as `sudo`, `su`, `passwd` and `pkexec`; the remaining 26 entries sit inside container image layers (EV-15). The app container holds eight standard SUID binaries (EV-16). Nothing unusual was found. The case folder listing shows every evidence file as `r--r--r--`; the register, hash file and chain of custody are shown in S09a to S09d.

7. Evidence register

Table 3 is generated from `evidence_register.csv`. All items were collected by the investigator on 1 November 2026. Only the first 16 characters of each SHA-256 are shown; full values are in Appendix C.

ID	File	Description	Source	UTC	Bytes	SHA-256 (first 16)
EV-01	system_info.txt	System time, host and Docker version	VM host	14:20:22	836	`00c5c93a7ab987fb`
EV-02	network_connections.txt	Listening ports and active connections on the VM	VM host	14:20:22	2611	`9ffee65f60610622`

ID	File	Description	Source	UTC	Bytes	SHA-256 (first 16)
EV-03	app_network_connections.txt	Listening ports and connections inside the app container	securebank-app container	14:20:23	643	3bda63cd8405654d
EV-04	network_interfaces.txt	Network interfaces and IP addresses on the VM	VM host	14:20:23	3473	28a77515da7d01a7
EV-05	container_network.txt	Docker lab network with each container IP	Docker engine	14:20:23	3853	f9b2d2121559b12c
EV-06	running_processes.txt	All running processes on the VM	VM host	14:20:23	18597	4f891728d1b433f3
EV-07	container_processes.txt	Processes running inside each lab container	Docker engine	14:20:23	7921	e52d486a4524b926
EV-08	lab_containers.txt	Lab containers, images, status and published ports	Docker engine	14:20:23	1024	b5fd013dba810580
EV-09	securebank.log	SecureBank application security log	lab/evidence/ securebank.log	14:20:23	3861	81f65a7107e24506
EV-10	app_container_output.txt	App container console output with Docker timestamps	securebank-app container	14:20:23	289	75684c8ca72df4a8
EV-11	attack_timeline.txt	Attack timeline written by the lab attack script	lab/evidence/ attack_timeline.txt	14:20:23	98	fc7dd97f0a6cbbc7
EV-12	log_files.txt	Log files present on the VM file system	VM host	14:20:23	9107	5ef73ce91b2eac00
EV-13	file_permissions.txt	Permissions and owners of every lab file	VM host	14:20:25	4383	fb511514bfc887df
EV-14	app_file_permissions.txt	Permissions of application files inside the container	securebank-app container	14:20:25	884	d5fbd373dc91c3a3
EV-15	suid_binaries.txt	SUID binaries on the VM	VM host	14:20:25	4744	14963132ccda3105
EV-16	app_suid_binaries.txt	SUID binaries inside the app container	securebank-app container	14:20:25	701	99afeb07518e8e04
EV-17	disk_usage.txt	Disk usage of the VM and Docker	VM host	14:20:26	773	b501dfab989dab71
EV-18	config_references.txt	Lab version, local changes, and config lines with debug mode, secrets or credentials	Lab repository	14:20:26	2001	80b49b357b009d4f
EV-19	docker_inspect_app.json	Full Docker configuration of the app container	Docker engine	14:20:26	9744	53497c843e95e39e
EV-20	config/docker-compose.yml	Lab Docker Compose file	lab/docker-compose.yml	14:20:26	2266	f673311b9b756d9c
EV-21	config/app.py	Application source code	lab/vulnerable-app/ app.py	14:20:26	3811	1b2cbe4f5d9b1772
EV-22	config/db_init.sql	Database initialisation script	lab/vulnerable-app/ db_init.sql	14:20:26	768	4d0699b88feb825e
EV-23	config/app.Dockerfile	App container build file	lab/vulnerable-app/Dockerfile	14:20:26	446	c903a0f396d9a4eb
EV-24	config/logstash.conf	Logstash pipeline configuration	lab/elk-stack/ logstash.conf	14:20:26	981	7b176c5df474816f

Table 3: Evidence register ("lab/" = ~/secure-cyber-fortress/). Source: evidence_register.csv, S09b

8. Hash verification and chain of custody

Table 4 reproduces the chain of custody. Entries 1 and 2 were written by the script, entry 3 after verification in the VM, and entry 4 after transfer to the Mac.

#	Date (UTC)	Item	Action	From → To	Hash status
1	14:20:22	EV-01 to EV-24	Collected	securebank-lab VM and lab containers → ~/case_SB-2026-001/evidence	SHA-256 and MD5 recorded in hashes.txt
2	14:20:26	SB-2026-001_evidence.tar.gz	Sealed read-only and archived	~/case_SB-2026-001/evidence → ~/case_SB-2026-001	Archive SHA-256 8cc882af5a645e59…
3	14:21:56	EV-01 to EV-24	Integrity verified	~/case_SB-2026-001 → ~/case_SB-2026-001	All SHA-256 hashes OK
4	14:38:30	Case package SB-2026-001	Transferred by multipass transfer	securebank-lab VM → Mac ~/Downloads/8321	Package hash matched; all evidence SHA-256 OK on Mac

Table 4: Chain of custody (handled by the investigator throughout). Source: chain_of_custody.csv, S09d

Verification in the VM (S10). `sha256sum -c` reported `OK` for all 24 files, and the archive's hash matched. Writing to `evidence/securebank.log` failed with `Permission denied`, which shows the seal works.

Tamper test (S10). A copy of `securebank.log` was made in `/tmp`. The original and the copy both hashed to `81f65a71…d34cc`. After one line, `FAKE LOGIN_SUCCESS username=admin`, was added to the copy, the copy's hash became `b2e49d3e…c61c9` while the original's stayed the same. Because `cp` keeps the read-only mode, the copy needed `chmod u+w` before the line could be added; this was applied only to the copy and recorded in the logbook.

Verification after transfer (S11). The case package hashed to `f43cef12…195ecf` both in the VM and on the Mac, and all 24 evidence files returned `OK` with `shasum -a 256 -c` on the Mac.

How integrity was preserved. Evidence was collected into a case folder separate from the live lab directory, so no running process could alter it. Volatile data was captured first, in line with RFC 3227. Each item was hashed with SHA-256 as the primary check and MD5 as a secondary one, kept for compatibility despite MD5's known collision weakness. The folder was then made read-only and archived, and the archive was hashed. Hashes were re-verified after sealing and again after transfer to the analysis computer; all matched. Every action was recorded with its UTC time in the collection log, the logbook and the chain of custody.

9. Observations and recommendations

Table 5 lists only weaknesses the collected evidence confirms, each with a control linked to it.

Observation	Evidence	Recommended control
SQL-injection string accepted as a login: `admin' OR '1'='1` logged as `LOGIN_SUCCESS` from 172.18.0.7 at 14:17:28Z. The login query is built by string formatting (line 36).	EV-09, EV-21, S07	Parameterised queries; input validation
Flask debug mode on (line 117); log records `Debugger is active!` and the debugger PIN.	EV-09, EV-10, EV-18, EV-21	Disable debug mode; run under a production WSGI server
Hard-coded secrets: `MYSQL_ROOT_PASSWORD=root`, `MYSQL_PASSWORD=password`, `secret_key = 'insecure_secret_key_123'`; the password also appears in the container environment.	EV-18, EV-19, EV-20, EV-21	Move secrets to a secrets manager or an untracked `.env`; rotate them
Passwords stored in plain text in the `users` table.	EV-22	Salted hashing with bcrypt or Argon2
Three failed logins for jane, then success, with no lockout or alert (activity generated in this test).	EV-09, S07	Account lockout, MFA, real-time alerting
Search input reflected without escaping; XSS payload request returned HTTP 200.	EV-09, EV-21	Output encoding and a Content Security Policy
Ports 8080, 9200, 5601 and 5000 published on all interfaces (`0.0.0.0`).	EV-02, EV-08	Bind to `127.0.0.1` or restrict with a firewall
Logstash reads `/logs/securebank.log` (mounted from `evidence/logs/`), but the app writes to `evidence/securebank.log`; `evidence/logs/` holds no files, so events never reach ELK.	EV-12, EV-13, EV-20, EV-21, EV-24	Fix the pipeline path; centralised log monitoring
Kali container runs `privileged` with the Docker socket mounted; the app runs as root.	EV-07, EV-19, EV-20	Least privilege: drop privileged mode, non-root users

Table 5: Observations and linked controls

10. Limitations

- Live acquisition changes the system slightly: the collection script and `ps` itself appear in the process list (EV-06), and the investigator's own requests appear in the log from 172.18.0.1 (EV-09).

- No memory image or disk image was taken; container logs would be lost if the containers were removed.
- MD5 has known collision weaknesses and is secondary only; SHA-256 is the integrity control relied on.
- There was a single investigator and no independent witness.
- `attack_timeline.txt` (EV-11) is written by the lecturer's script with a hard-coded attacker IP of 172.16.0.100; the log shows the requests actually came from 172.18.0.7 (EV-05, EV-09).
- The command-injection test was redirected (HTTP 302) because it had no session (EV-09). Whether `os.popen` would execute the input is known only from the code (EV-21), not from a test.
- Logs and the register use UTC. The VM's local time zone is Africa/Lagos, one hour ahead of UTC (EV-01), so `ls` and `find` timestamps in EV-12, EV-13 and S09a are local time.
- Screenshots S01 to S05 were captured between 14:10Z and 14:15Z, after the lab was started (13:13Z, EV-10) and the case opened (13:15:46Z), so they show the state at capture time.

11. Conclusion

Twenty-four evidence items were collected from the SecureBank lab in order of volatility, hashed, registered, sealed read-only and archived. Their integrity was proven twice: in the VM after sealing and on the analysis computer after transfer. A complete four-entry chain of custody records every handling step. Because each item can be re-checked against `hashes.txt`, the evidence is fit for the log analysis, timeline reconstruction and remediation work in the other lab exercises. It already supports specific findings on SQL injection, debug mode, hard-coded secrets and broken log shipping.

12. Reflection

This exercise changed how I think about evidence. I used to see hashing as a box to tick, but watching one added line give my working copy a completely different hash, while the original stayed at `81f65a71…` (S10), showed me why a recorded hash is real proof of integrity. Capturing network sockets and processes first, all within about a second, made the RFC 3227 order of volatility concrete rather than theoretical. I also learned not to trust a single source: `attack_timeline.txt` named 172.16.0.100 as the attacker, but the application log showed the requests came from 172.18.0.7. And I now see why investigators standardise on UTC, because my VM's file listings were in Lagos time while the logs were not.

The hardest part was the environment rather than the forensics. macOS blocked access to the VM until I granted Local Network and Screen Recording permissions, MySQL had to be changed to version 8.0 for my ARM Mac, and the guide's step for starting the lab was blank. I also missed the screenshots for the first two steps and had to capture them afterwards. Small surprises mattered too: the app had no logout route, `cp` kept the read-only mode so the tamper test needed `chmod` on the copy, and the ELK stack never received the app's logs because of a path mismatch.

Next time I would take every screenshot and logbook entry at the moment of the action, hash the collection script before running it, and ask a second person to witness and co-sign the chain of custody. I would add a memory capture and an export of the container logs, rely on SHA-256 alone, and fix the Logstash path first so the events could also be checked in Kibana.

Appendix A: Collection script and lab configuration change

A.1 collect_evidence.sh

This appendix contains the complete code base for this exercise. The same script, with the evidence folder, register, hash files, chain of custody, logs and screenshots, is in the GitHub repository https://github.com/obiwanpelosi/mit8321-ca-exercise4 at `case_SB-2026-001/scripts/collect_evidence.sh`. The script was saved unchanged from the Step 4 instructions; its SHA-256 is `5d3ccf3b3ba025e5dc5b419f84d88b9f89db2b1db7aa46451cd019a0411b1e03` (logbook, Appendix E).

```
#!/usr/bin/env bash
# collect_evidence.sh - Evidence collection and preservation
# MIT8321 CA, Lab Exercise 4 - Case SB-2026-001
# Order: volatile data first (RFC 3227 order of volatility), then logs,
# file-system and configuration data. Every item is hashed (SHA-256 + MD5),
# listed in an evidence register, recorded in a chain-of-custody file,
# then sealed read-only and archived.
set -u

CASE_ID="SB-2026-001"
INVESTIGATOR="Feyi Abioye"
LAB_DIR="$HOME/secure-cyber-fortress"
CASE_DIR="$HOME/case_${CASE_ID}"
EV="$CASE_DIR/evidence"
LOG="$CASE_DIR/collection_log.txt"
REGISTER="$CASE_DIR/evidence_register.csv"
COC="$CASE_DIR/chain_of_custody.csv"
APP="securebank-app"

ts()  { date -u +%Y-%m-%dT%H:%M:%SZ; }
log() { echo "$(ts) | $INVESTIGATOR | $*" | tee -a "$LOG"; }

# Never overwrite evidence that already exists
if [ -d "$EV" ] && [ -n "$(ls -A "$EV" 2>/dev/null)" ]; then
  echo "ERROR: $EV already holds evidence. Refusing to overwrite it."
  exit 1
fi
mkdir -p "$EV/config"

declare -a ORDER=()
declare -A DESC SRC WHEN

# capture OUTFILE "description" "source" command [args...]
capture() {
  local out="$1" desc="$2" src="$3"; shift 3
  WHEN[$out]="$(ts)"; DESC[$out]="$desc"; SRC[$out]="$src"; ORDER+=("$out")
  log "COLLECT $out <= $*"
  {
    echo "# Case: $CASE_ID | Collected: ${WHEN[$out]} | By: $INVESTIGATOR | Host: $(hostname)"
```

```
    echo "# Source: $src"
    echo "# Command: $*"
    echo
    "$@" 2>&1
  } > "$EV/$out"
}

# copy_file SOURCE_PATH OUTFILE "description"  (cp -p keeps original timestamps)
copy_file() {
  local from="$1" out="$2" desc="$3"
  if [ -f "$from" ]; then
    WHEN[$out]="$(ts)"; DESC[$out]="$desc"; SRC[$out]="$from"; ORDER+=("$out")
    cp -p "$from" "$EV/$out"
    log "COPY $out <= $from"
  else
    log "SKIP $out (not found: $from)"
  fi
}

log "=== Collection started for case $CASE_ID on $(hostname) ==="

# 1. Volatile data: system state, network, processes
capture system_info.txt "System time, host and Docker version" "VM host" \
  bash -c 'date -u; echo; hostname; uname -a; echo; uptime; echo; timedatectl 2>/dev/null; echo; docker version --format "Docker server
{{.Server.Version}}"'
capture network_connections.txt "Listening ports and active connections on the VM" "VM host" \
  sudo ss -tulpan
capture app_network_connections.txt "Listening ports and connections inside the app container" "$APP container" \
  docker exec "$APP" netstat -tulpan
capture network_interfaces.txt "Network interfaces and IP addresses on the VM" "VM host" \
  ip addr show
capture container_network.txt "Docker lab network with each container IP" "Docker engine" \
  bash -c 'docker network inspect $(docker network ls -q --filter name=securebank-net)'
capture running_processes.txt "All running processes on the VM" "VM host" \
  ps auxww --sort=start_time
capture container_processes.txt "Processes running inside each lab container" "Docker engine" \
  bash -c 'for c in $(docker ps --format "{{.Names}}"); do echo "=== $c ==="; docker top "$c"; echo; done'
capture lab_containers.txt "Lab containers, images, status and published ports" "Docker engine" \
  docker ps -a --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'

# 2. Logs
copy_file "$LAB_DIR/evidence/securebank.log" securebank.log "SecureBank application security log"
capture app_container_output.txt "App container console output with Docker timestamps" "$APP container" \
  docker logs --timestamps "$APP"
copy_file "$LAB_DIR/evidence/attack_timeline.txt" attack_timeline.txt "Attack timeline written by the lab attack script"
capture log_files.txt "Log files present on the VM file system" "VM host" \
  bash -c "sudo find / -xdev -type f -name '*.log' -printf '%TY-%Tm-%Td %TH:%TM  %10s bytes  %p\n' 2>/dev/null | sort"

# 3. File system: permissions, SUID binaries, disk usage
capture file_permissions.txt "Permissions and owners of every lab file" "VM host" \
  bash -c "find '$LAB_DIR' -path '*/.git' -prune -o -printf '%M %u:%g %8s %TY-%Tm-%Td %TH:%TM %p\n'"
capture app_file_permissions.txt "Permissions of application files inside the container" "$APP container" \
  docker exec "$APP" ls -la /app /app/evidence
capture suid_binaries.txt "SUID binaries on the VM" "VM host" \
  bash -c 'sudo find / -xdev -perm -4000 -type f -exec ls -l {} + 2>/dev/null'
```

```
capture app_suid_binaries.txt "SUID binaries inside the app container" "$APP container" \
  docker exec "$APP" sh -c 'find / -xdev -perm -4000 -type f -exec ls -l {} + 2>/dev/null'
capture disk_usage.txt "Disk usage of the VM and Docker" "VM host" \
  bash -c 'df -h; echo; docker system df'

# 4. Configuration
capture config_references.txt "Lab version, local changes, and config lines with debug mode, secrets or credentials" "Lab repository" \
  bash -c "cd '$LAB_DIR' && git log -1 --format='Commit %H (%ci)' && echo && git status --short && git diff -- docker-compose.yml vulnerable-app && echo &&
grep -nE 'debug=True|secret_key|PASSWORD|password' docker-compose.yml vulnerable-app/app.py vulnerable-app/db_init.sql"
capture docker_inspect_app.json "Full Docker configuration of the app container" "Docker engine" \
  docker inspect "$APP"
copy_file "$LAB_DIR/docker-compose.yml"          config/docker-compose.yml   "Lab Docker Compose file"
copy_file "$LAB_DIR/vulnerable-app/app.py"       config/app.py               "Application source code"
copy_file "$LAB_DIR/vulnerable-app/db_init.sql"  config/db_init.sql          "Database initialisation script"
copy_file "$LAB_DIR/vulnerable-app/Dockerfile"   config/app.Dockerfile       "App container build file"
copy_file "$LAB_DIR/elk-stack/logstash.conf"     config/logstash.conf        "Logstash pipeline configuration"

# 5. Hash every evidence file (paths relative to evidence/ so they can be re-checked)
log "Hashing all evidence files (SHA-256 and MD5)"
( cd "$EV" && find . -type f -print0 | sort -z | xargs -0 sha256sum ) > "$CASE_DIR/hashes.txt"
( cd "$EV" && find . -type f -print0 | sort -z | xargs -0 md5sum )    > "$CASE_DIR/hashes_md5.txt"

# 6. Evidence register
echo "Evidence_ID,File,Description,Source,Collected_by,Collected_UTC,Size_bytes,SHA256" > "$REGISTER"
n=0
for out in "${ORDER[@]}"; do
  n=$((n + 1))
  size=$(stat -c %s "$EV/$out")
  sha=$(sha256sum "$EV/$out" | cut -d' ' -f1)
  printf 'EV-%02d,%s,"%s","%s",%s,%s,%s,%s\n' "$n" "$out" "${DESC[$out]}" "${SRC[$out]}" \
    "$INVESTIGATOR" "${WHEN[$out]}" "$size" "$sha" >> "$REGISTER"
done
log "Evidence register written: $n items"

# 7. Seal: make evidence read-only, archive it, hash the archive
chmod -R a-w "$EV"
tar -czf "$CASE_DIR/${CASE_ID}_evidence.tar.gz" -C "$CASE_DIR" evidence
( cd "$CASE_DIR" && sha256sum "${CASE_ID}_evidence.tar.gz" > "${CASE_ID}_evidence.tar.gz.sha256" )
chmod a-w "$CASE_DIR/hashes.txt" "$CASE_DIR/hashes_md5.txt" "$REGISTER"
ARCHIVE_SHA=$(cut -d' ' -f1 "$CASE_DIR/${CASE_ID}_evidence.tar.gz.sha256")
log "Evidence sealed read-only. Archive SHA-256: $ARCHIVE_SHA"

# 8. Chain of custody: first two entries (later entries are added by hand)
cat > "$COC" <<COC
Entry,Date_UTC,Item,Action,From,To,Handled_by,Purpose,Hash_status
1,${WHEN[${ORDER[0]}]},EV-01 to EV-$(printf '%02d' "$n"),Collected,securebank-lab VM and lab containers,$EV,$INVESTIGATOR,Evidence acquisition,SHA-256 and
MD5 recorded in hashes.txt
2,$(ts),${CASE_ID}_evidence.tar.gz,Sealed read-only and archived,$EV,$CASE_DIR,$INVESTIGATOR,Preservation,Archive SHA-256 $ARCHIVE_SHA
COC

log "=== Collection finished: $n items. Case folder: $CASE_DIR ==="
```

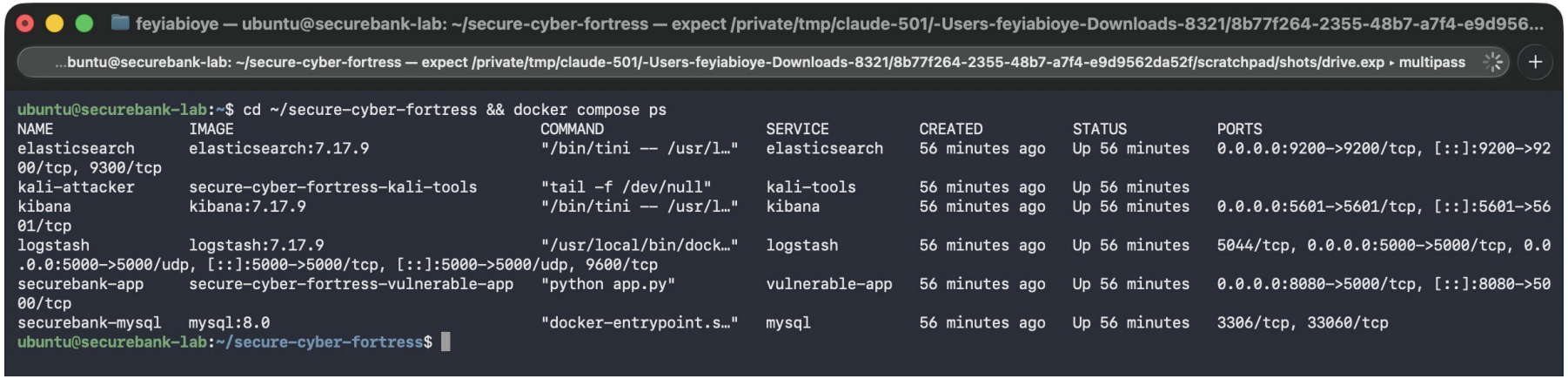
A.2 docker-compose.yml change (from EV-18)

```
diff --git a/docker-compose.yml b/docker-compose.yml
index f615c9e..ba7e8b4 100644
--- a/docker-compose.yml
+++ b/docker-compose.yml
@@ -19,7 +19,7 @@ services:

   # MySQL Database
   mysql:
-    image: mysql:5.7
+    image: mysql:8.0
     container_name: securebank-mysql
     environment:
       - MYSQL_ROOT_PASSWORD=root
```

Appendix B: Screenshots S01–S12

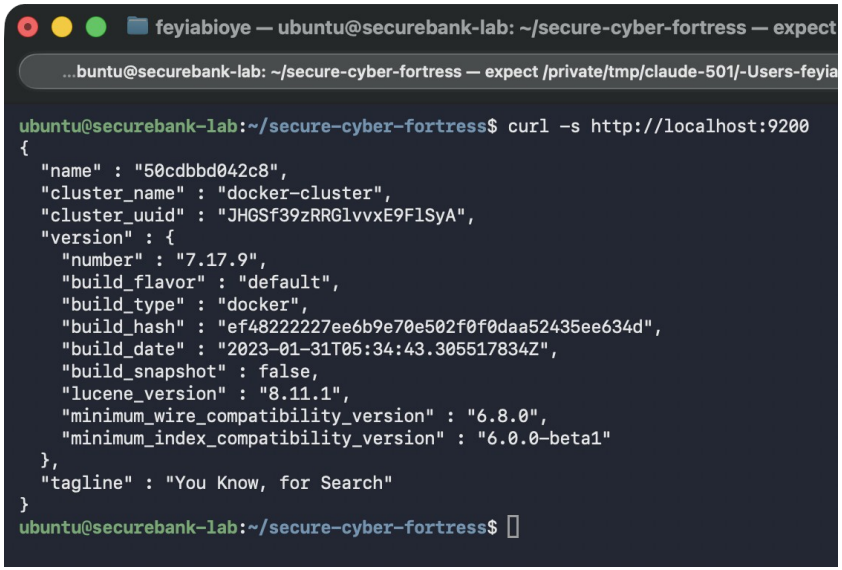
Captured from real Terminal windows: S01 to S10 in a `multipass shell` session on the VM, S11 and S12 on the Mac. Times are UTC. S09 is split into four parts, one per command. Empty terminal background below and to the right of the output has been trimmed for readability; the uncropped originals are in the repository's `screenshots/` folder.



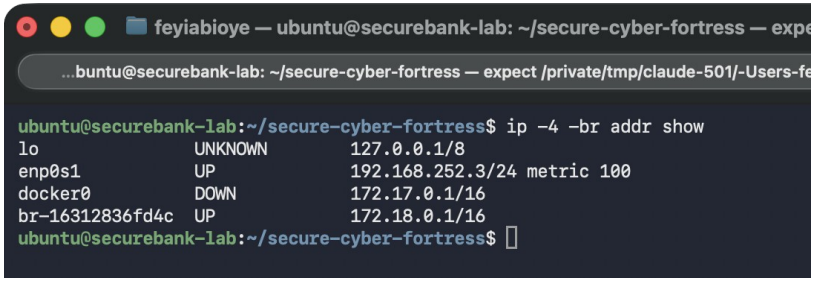
S01 (14:10:42Z): docker compose ps showing all six lab containers running



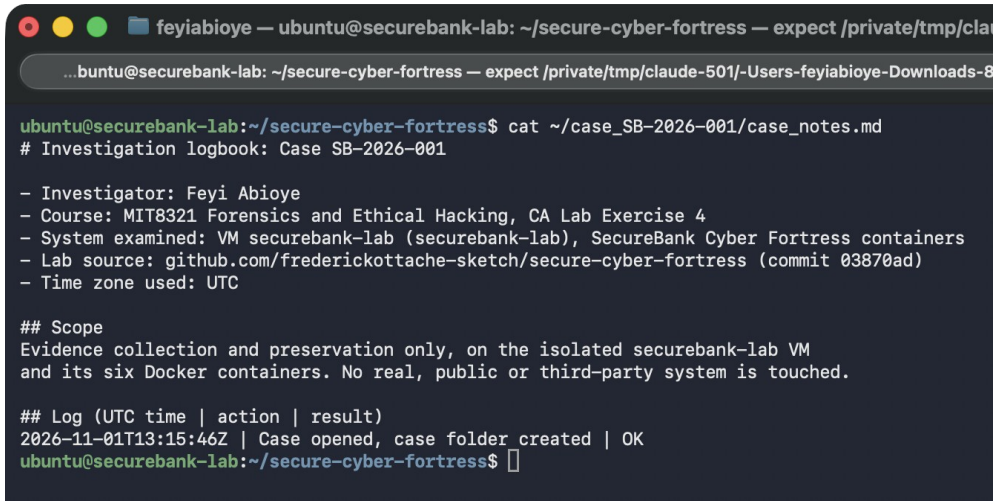
S02 (14:12:20Z): SecureBank page HTML returned by curl localhost:8080 inside the VM



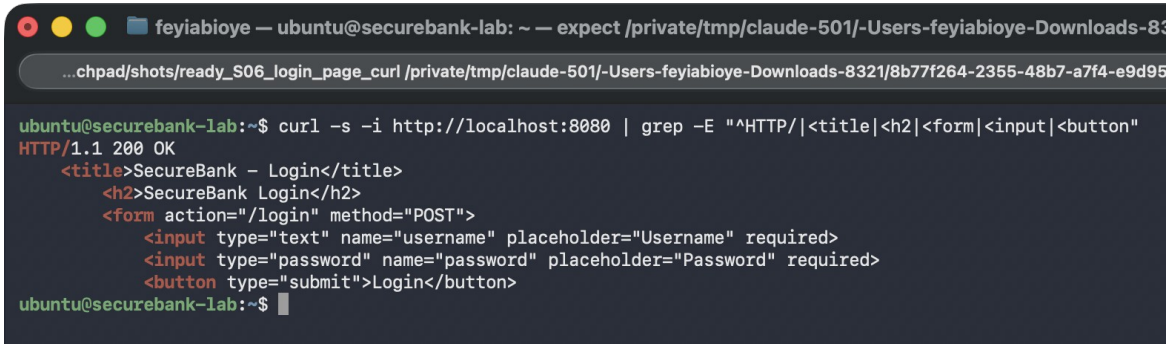
S03 (14:12:50Z): Elasticsearch 7.17.9 version JSON from curl localhost:9200



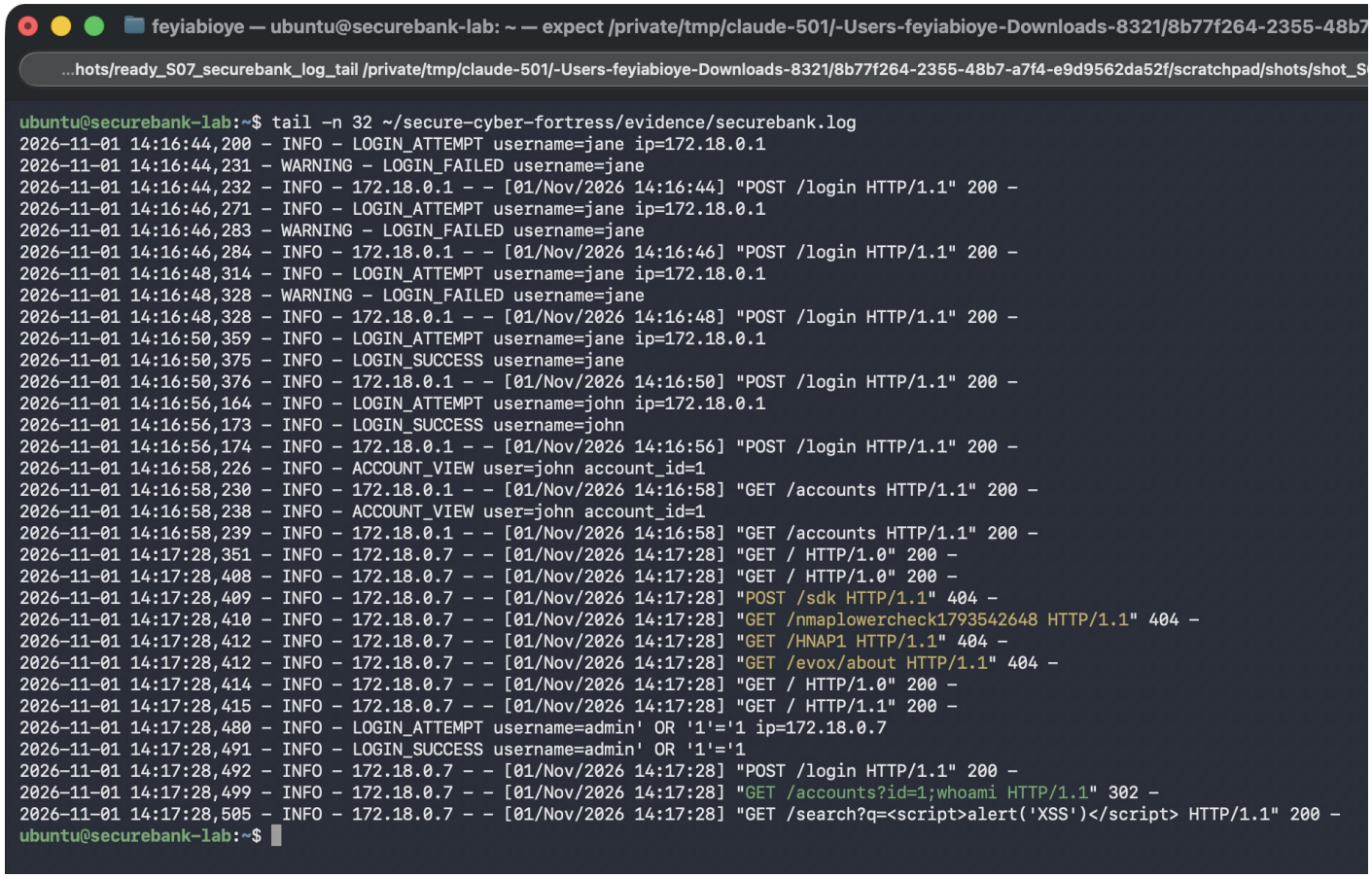
S04 (14:13:50Z): VM interfaces; enp0s1 is 192.168.252.3, lab bridge 172.18.0.1



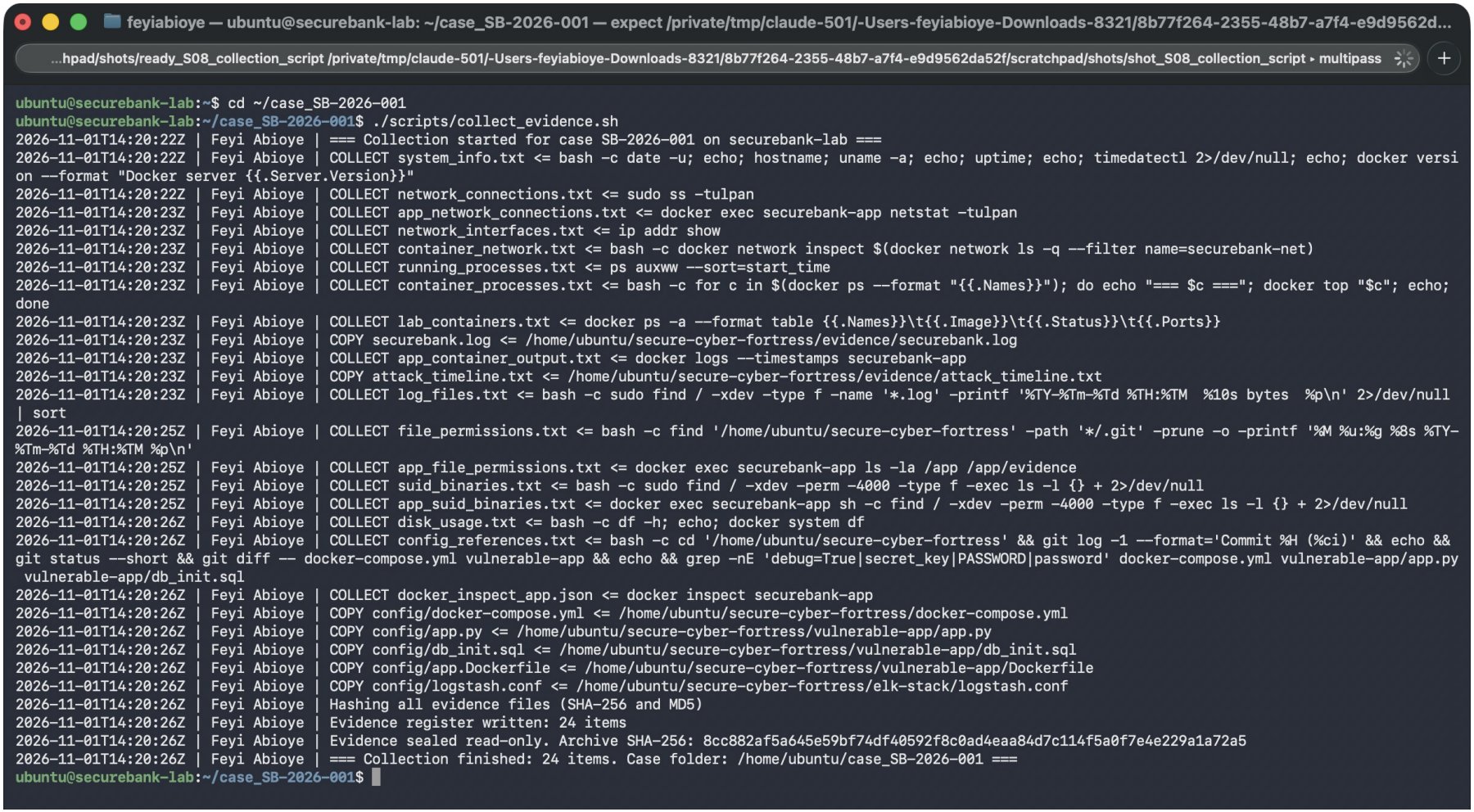
S05 (14:14:58Z): Investigation logbook just after the case was opened



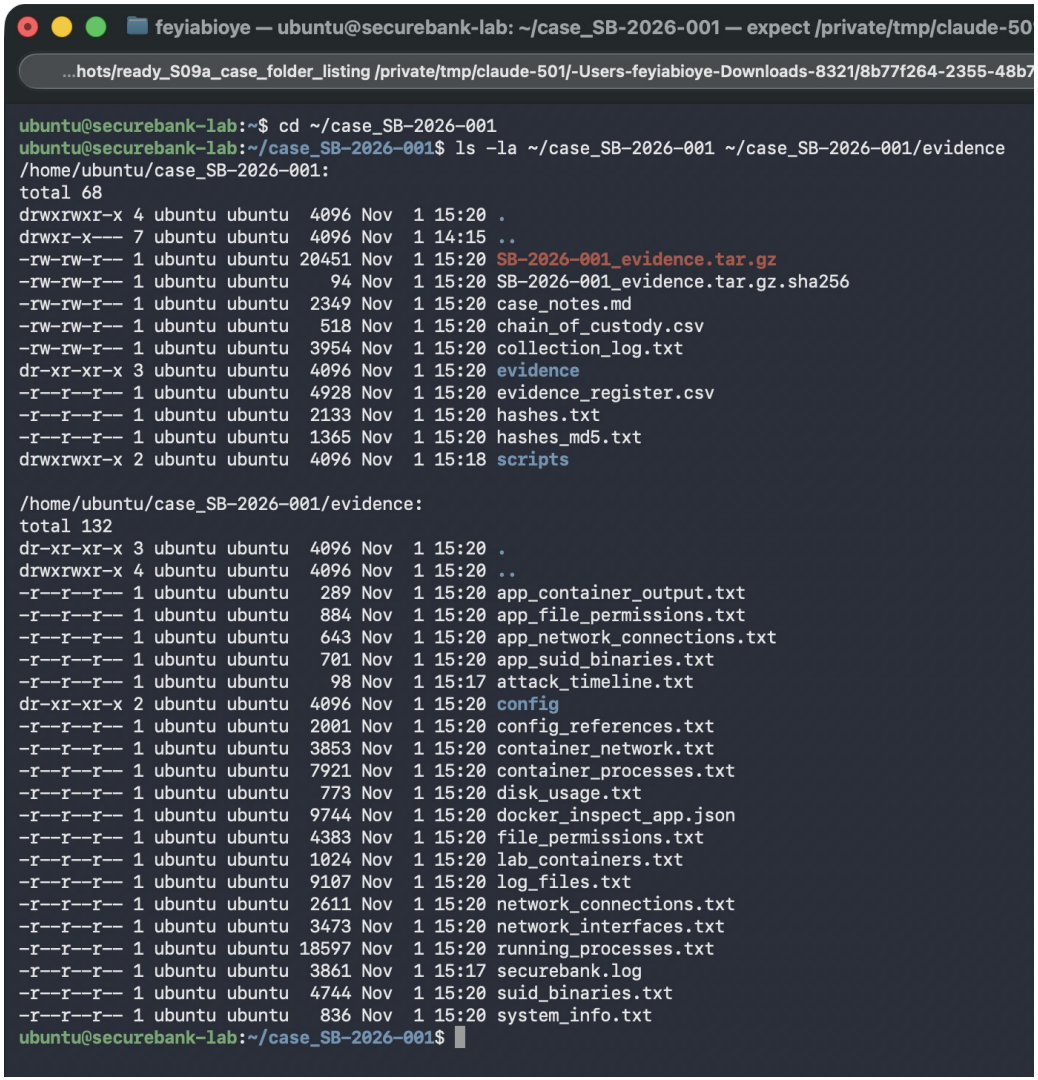
S06 (14:16:32Z): SecureBank login form fetched with curl inside the VM (HTTP 200)



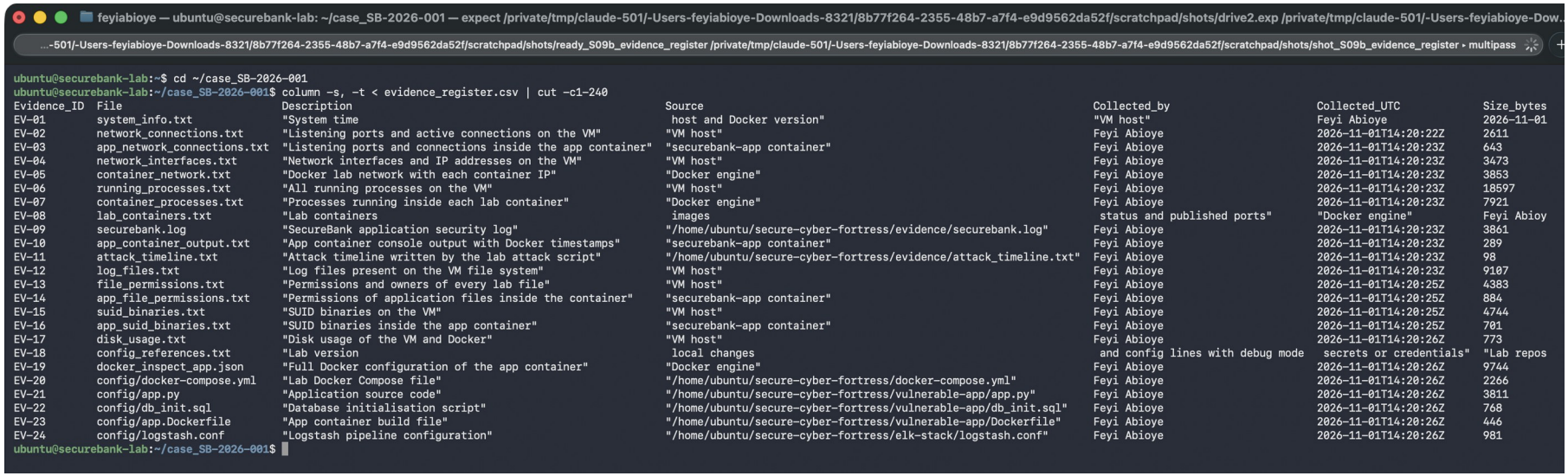
S07 (14:17:46Z): tail -n 32 of securebank.log showing jane's failures and success, john's activity, and the attack-script requests



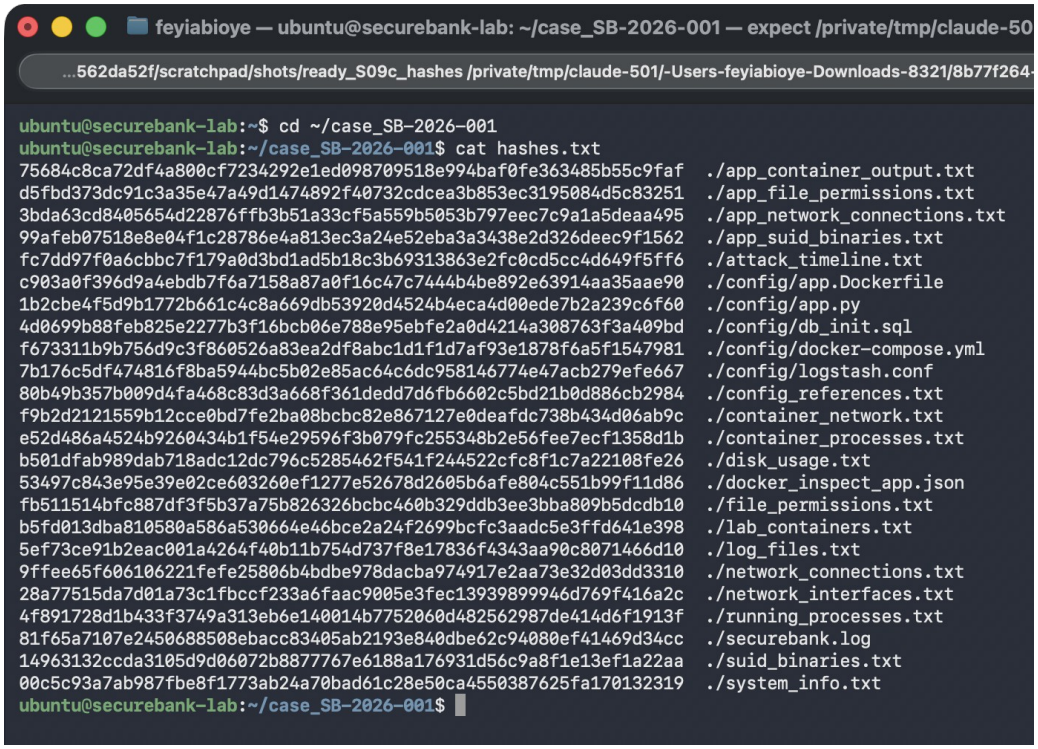
S08 (14:20:27Z): collect_evidence.sh completing with 24 items, sealed and archived



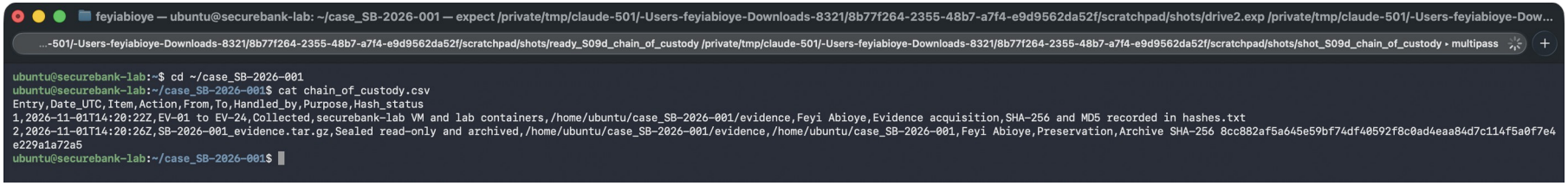
S09a (14:21:06Z): Case folder and read-only evidence folder



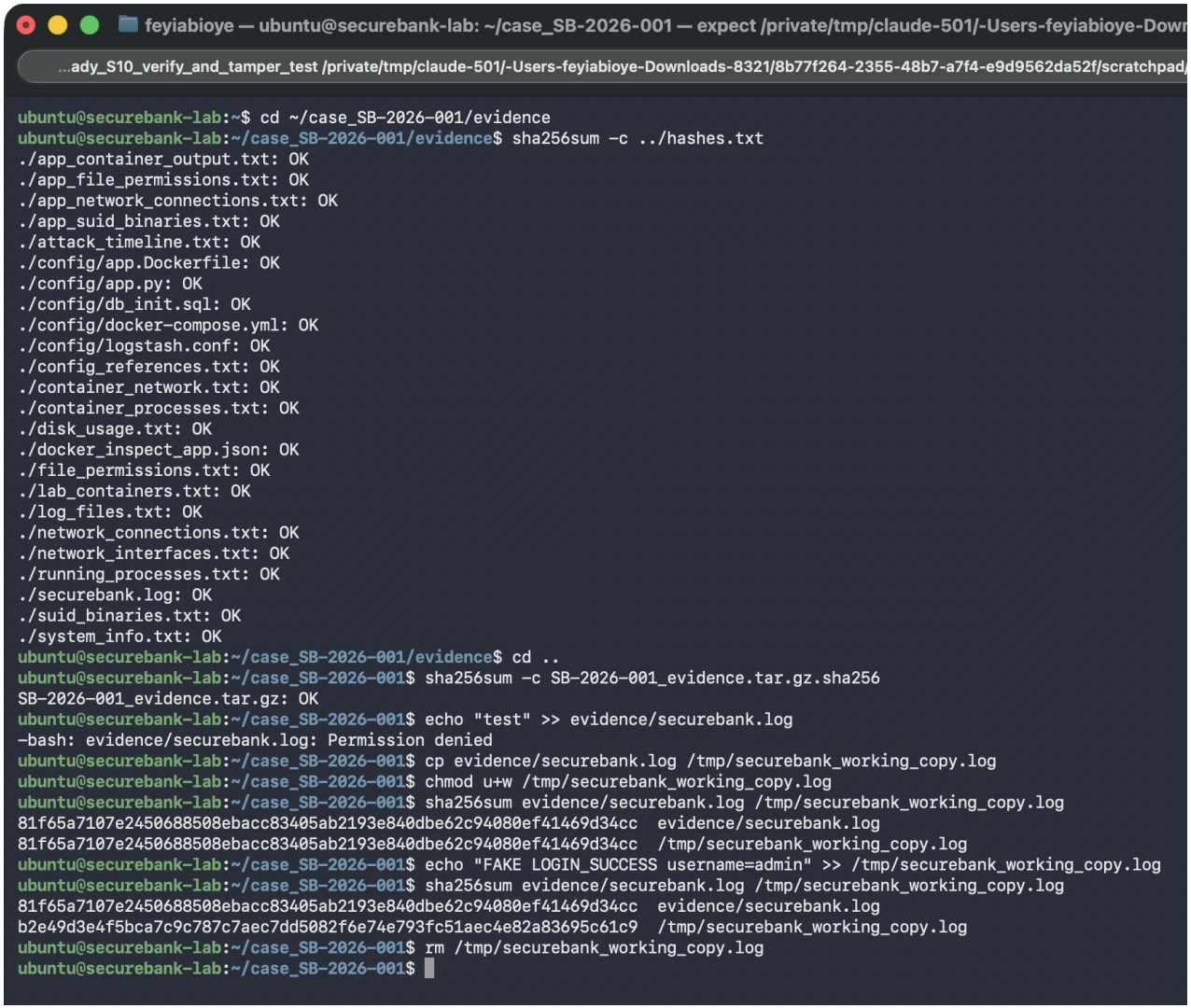
S09b (14:21:08Z): Evidence register (column -t, cut to 240 characters; commas in three descriptions shift those rows)



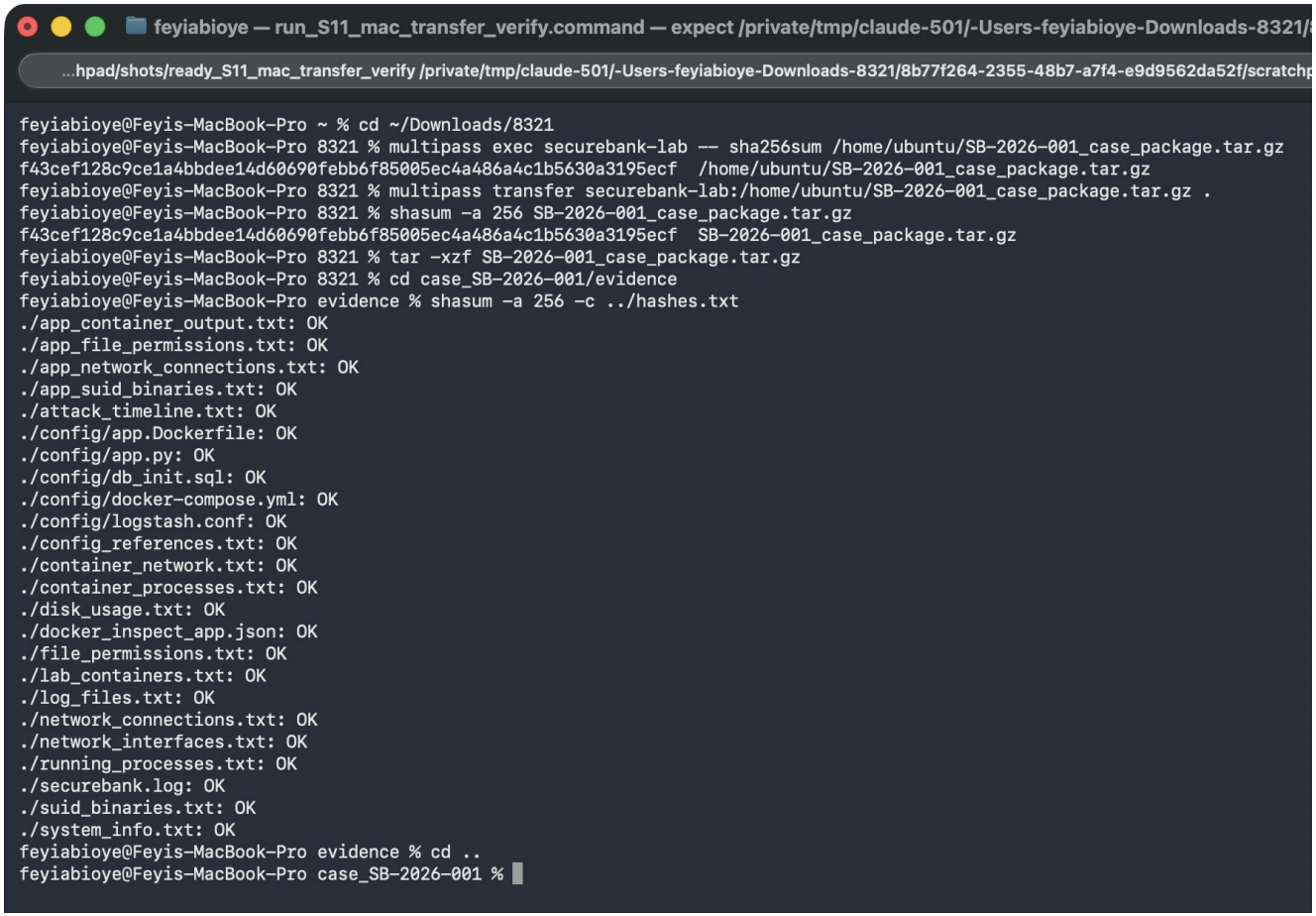
S09c (14:21:11Z): hashes.txt with SHA-256 of all 24 items



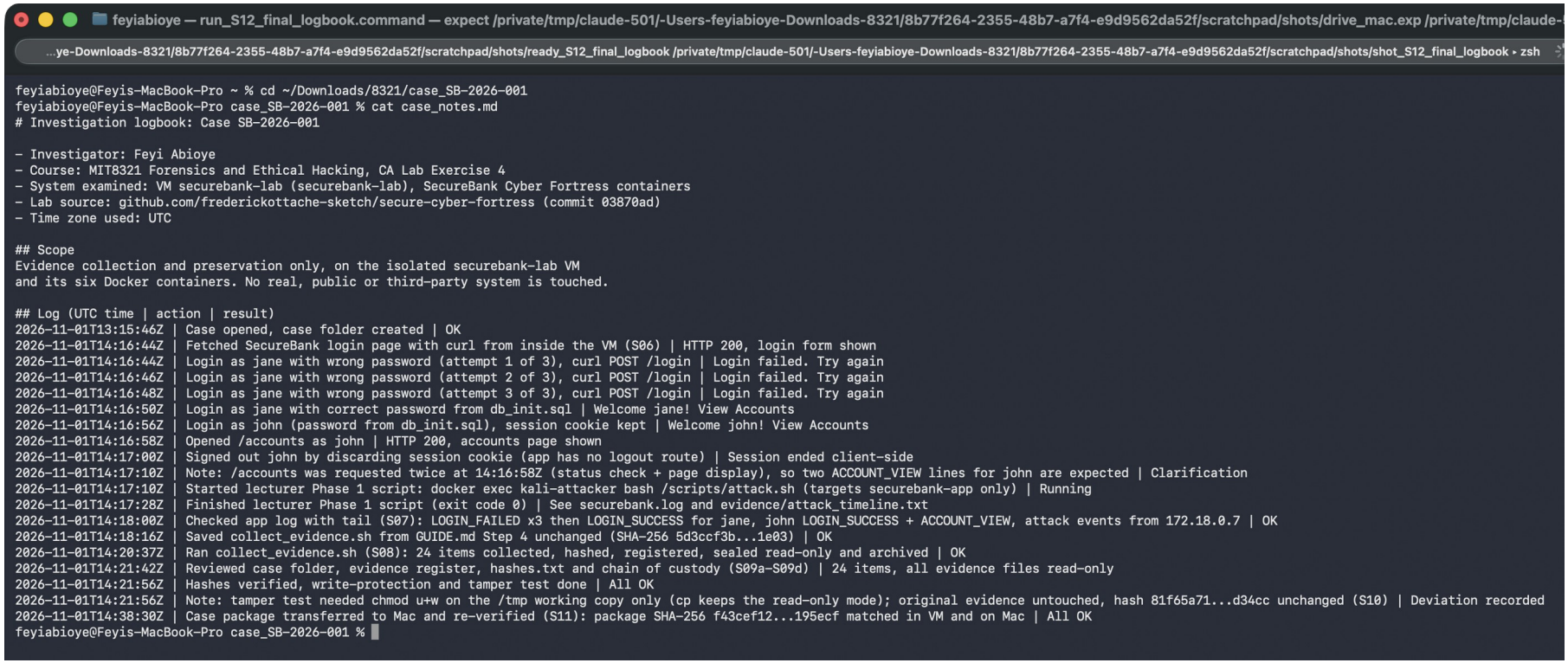
S09d (14:21:13Z): Chain of custody entries 1 (collected) and 2 (sealed)



S10 (14:21:44Z): All hashes OK, write blocked with Permission denied, and tamper test on a /tmp working copy



S11 (14:38:22Z): Matching package hash in the VM and on the Mac, and all-OK verification on the Mac



S12 (14:38:40Z): Final investigation logbook

Appendix C: Hash values

C.1 hashes.txt (SHA-256)

```
75684c8ca72df4a800cf7234292e1ed098709518e994baf0fe363485b55c9faf  ./app_container_output.txt
d5fbd373dc91c3a35e47a49d1474892f40732cdcea3b853ec3195084d5c83251  ./app_file_permissions.txt
3bda63cd8405654d22876ffb3b51a33cf5a559b5053b797eec7c9a1a5deaa495  ./app_network_connections.txt
99afeb07518e8e04f1c28786e4a813ec3a24e52eba3a3438e2d326deec9f1562  ./app_suid_binaries.txt
fc7dd97f0a6cbbc7f179a0d3bd1ad5b18c3b69313863e2fc0cd5cc4d649f5ff6  ./attack_timeline.txt
c903a0f396d9a4ebdb7f6a7158a87a0f16c47c7444b4be892e63914aa35aae90  ./config/app.Dockerfile
1b2cbe4f5d9b1772b661c4c8a669db53920d4524b4eca4d00ede7b2a239c6f60  ./config/app.py
4d0699b88feb825e2277b3f16bcb06e788e95ebfe2a0d4214a308763f3a409bd  ./config/db_init.sql
f673311b9b756d9c3f860526a83ea2df8abc1d1f1d7af93e1878f6a5f1547981  ./config/docker-compose.yml
7b176c5df474816f8ba5944bc5b02e85ac64c6dc958146774e47acb279efe667  ./config/logstash.conf
80b49b357b009d4fa468c83d3a668f361dedd7d6fb6602c5bd21b0d886cb2984  ./config_references.txt
f9b2d2121559b12cce0bd7fe2ba08bcbc82e867127e0deafdc738b434d06ab9c  ./container_network.txt
e52d486a4524b9260434b1f54e29596f3b079fc255348b2e56fee7ecf1358d1b  ./container_processes.txt
b501dfab989dab718adc12dc796c5285462f541f244522cfc8f1c7a22108fe26  ./disk_usage.txt
53497c843e95e39e02ce603260ef1277e52678d2605b6afe804c551b99f11d86  ./docker_inspect_app.json
fb511514bfc887df3f5b37a75b826326bcbc460b329ddb3ee3bba809b5dcdb10  ./file_permissions.txt
b5fd013dba810580a586a530664e46bce2a24f2699bcfc3aadc5e3ffd641e398  ./lab_containers.txt
5ef73ce91b2eac001a4264f40b11b754d737f8e17836f4343aa90c8071466d10  ./log_files.txt
9ffee65f606106221fefe25806b4bdbe978dacba974917e2aa73e32d03dd3310  ./network_connections.txt
28a77515da7d01a73c1fbccf233a6faac9005e3fec13939899946d769f416a2c  ./network_interfaces.txt
4f891728d1b433f3749a313eb6e140014b7752060d482562987de414d6f1913f  ./running_processes.txt
81f65a7107e2450688508ebacc83405ab2193e840dbe62c94080ef41469d34cc  ./securebank.log
14963132ccda3105d9d06072b8877767e6188a176931d56c9a8f1e13ef1a22aa  ./suid_binaries.txt
00c5c93a7ab987fbe8f1773ab24a70bad61c28e50ca4550387625fa170132319  ./system_info.txt
```

C.2 hashes_md5.txt (MD5)

```
5e359cc635d4b1bcc027b55efe76104d  ./app_container_output.txt
d5ea589f3260e8bc12f7003c46996f18  ./app_file_permissions.txt
05c03d9aeef9431acdfdad3165e81861  ./app_network_connections.txt
83315cf317b89a70834f071563286aff  ./app_suid_binaries.txt
62750fa2a8f7b016a6d35cdec0d5f08e  ./attack_timeline.txt
0995e5c0e17d195c6b8d85d8ac791fad  ./config/app.Dockerfile
cdb6f2f2f9caa79032e72fd9e7459adb  ./config/app.py
169f289e76addaa4b6143ae9cf29b278  ./config/db_init.sql
50451d865346a617b049e71ce9b5d760  ./config/docker-compose.yml
8723752e929884e0be0456e8280d2e8b  ./config/logstash.conf
a5fa0302bf67519c00ad49f921cdd5a2  ./config_references.txt
039af86a3a971090110b9ed1e9f2d64c  ./container_network.txt
fd6f274e1bab00cef91600b63a590933  ./container_processes.txt
0c5b34ac34d55158fa1b3e70ae3b98b5  ./disk_usage.txt
60b8c9dc17f3683d5c66809eebaf40fa  ./docker_inspect_app.json
316b037a39e0e45cb760d03395480bef  ./file_permissions.txt
d1c260431fa89e7b2f5e055fd664489a  ./lab_containers.txt
98f74a67634d45c02ea34c24eb65c5d7  ./log_files.txt
81d321e858ecb0af7df839036eb4414d  ./network_connections.txt
14da4e89d59511feb4b9509f76ee4ac1  ./network_interfaces.txt
97c020cc37dd89176dce8aa21ed095a1  ./running_processes.txt
```

```
2895742c472aa145ba2f69a0b60864ed  ./securebank.log
90443e7e7ee4ca49ad1d13bf92d74f39  ./suid_binaries.txt
49ca372afabe737216d0b4e0980fdab9  ./system_info.txt
```

C.3 Archive and transfer package

```
8cc882af5a645e59bf74df40592f8c0ad4eaa84d7c114f5a0f7e4e229a1a72a5  SB-2026-001_evidence.tar.gz
f43cef128c9ce1a4bbdee14d60690febb6f85005ec4a486a4c1b5630a3195ecf  SB-2026-001_case_package.tar.gz (VM and Mac, S11)
```

Appendix D: Collection log

```
2026-11-01T14:20:22Z | Feyi Abioye | === Collection started for case SB-2026-001 on securebank-lab ===
2026-11-01T14:20:22Z | Feyi Abioye | COLLECT system_info.txt <= bash -c date -u; echo; hostname; uname -a; echo; uptime; echo; timedatectl 2>/dev/null; echo; docker
version --format "Docker server {{.Server.Version}}"
2026-11-01T14:20:22Z | Feyi Abioye | COLLECT network_connections.txt <= sudo ss -tulpan
2026-11-01T14:20:23Z | Feyi Abioye | COLLECT app_network_connections.txt <= docker exec securebank-app netstat -tulpan
2026-11-01T14:20:23Z | Feyi Abioye | COLLECT network_interfaces.txt <= ip addr show
2026-11-01T14:20:23Z | Feyi Abioye | COLLECT container_network.txt <= bash -c docker network inspect $(docker network ls -q --filter name=securebank-net)
2026-11-01T14:20:23Z | Feyi Abioye | COLLECT running_processes.txt <= ps auxww --sort=start_time
2026-11-01T14:20:23Z | Feyi Abioye | COLLECT container_processes.txt <= bash -c for c in $(docker ps --format "{{.Names}}"); do echo "=== $c ==="; docker top "$c";
echo; done
2026-11-01T14:20:23Z | Feyi Abioye | COLLECT lab_containers.txt <= docker ps -a --format table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}
2026-11-01T14:20:23Z | Feyi Abioye | COPY securebank.log <= /home/ubuntu/secure-cyber-fortress/evidence/securebank.log
2026-11-01T14:20:23Z | Feyi Abioye | COLLECT app_container_output.txt <= docker logs --timestamps securebank-app
2026-11-01T14:20:23Z | Feyi Abioye | COPY attack_timeline.txt <= /home/ubuntu/secure-cyber-fortress/evidence/attack_timeline.txt
2026-11-01T14:20:23Z | Feyi Abioye | COLLECT log_files.txt <= bash -c sudo find / -xdev -type f -name '*.log' -printf '%TY-%Tm-%Td %TH:%TM  %10s bytes  %p\n'
2>/dev/null | sort
2026-11-01T14:20:25Z | Feyi Abioye | COLLECT file_permissions.txt <= bash -c find '/home/ubuntu/secure-cyber-fortress' -path '*/.git' -prune -o -printf '%M %u:%g %8s
%TY-%Tm-%Td %TH:%TM %p\n'
2026-11-01T14:20:25Z | Feyi Abioye | COLLECT app_file_permissions.txt <= docker exec securebank-app ls -la /app /app/evidence
2026-11-01T14:20:25Z | Feyi Abioye | COLLECT suid_binaries.txt <= bash -c sudo find / -xdev -perm -4000 -type f -exec ls -l {} + 2>/dev/null
2026-11-01T14:20:25Z | Feyi Abioye | COLLECT app_suid_binaries.txt <= docker exec securebank-app sh -c find / -xdev -perm -4000 -type f -exec ls -l {} + 2>/dev/null
2026-11-01T14:20:26Z | Feyi Abioye | COLLECT disk_usage.txt <= bash -c df -h; echo; docker system df
2026-11-01T14:20:26Z | Feyi Abioye | COLLECT config_references.txt <= bash -c cd '/home/ubuntu/secure-cyber-fortress' && git log -1 --format='Commit %H (%ci)' && echo
&& git status --short && git diff -- docker-compose.yml vulnerable-app && echo && grep -nE 'debug=True|secret_key|PASSWORD|password' docker-compose.yml vulnerable-
app/app.py vulnerable-app/db_init.sql
2026-11-01T14:20:26Z | Feyi Abioye | COLLECT docker_inspect_app.json <= docker inspect securebank-app
2026-11-01T14:20:26Z | Feyi Abioye | COPY config/docker-compose.yml <= /home/ubuntu/secure-cyber-fortress/docker-compose.yml
2026-11-01T14:20:26Z | Feyi Abioye | COPY config/app.py <= /home/ubuntu/secure-cyber-fortress/vulnerable-app/app.py
2026-11-01T14:20:26Z | Feyi Abioye | COPY config/db_init.sql <= /home/ubuntu/secure-cyber-fortress/vulnerable-app/db_init.sql
2026-11-01T14:20:26Z | Feyi Abioye | COPY config/app.Dockerfile <= /home/ubuntu/secure-cyber-fortress/vulnerable-app/Dockerfile
2026-11-01T14:20:26Z | Feyi Abioye | COPY config/logstash.conf <= /home/ubuntu/secure-cyber-fortress/elk-stack/logstash.conf
2026-11-01T14:20:26Z | Feyi Abioye | Hashing all evidence files (SHA-256 and MD5)
2026-11-01T14:20:26Z | Feyi Abioye | Evidence register written: 24 items
2026-11-01T14:20:26Z | Feyi Abioye | Evidence sealed read-only. Archive SHA-256: 8cc882af5a645e59bf74df40592f8c0ad4eaa84d7c114f5a0f7e4e229a1a72a5
2026-11-01T14:20:26Z | Feyi Abioye | === Collection finished: 24 items. Case folder: /home/ubuntu/case_SB-2026-001 ===
```

Appendix E: Investigation logbook

Final copy of `case_notes.md` from the Mac case folder (see also S12).

```
# Investigation logbook: Case SB-2026-001

- Investigator: Feyi Abioye
- Course: MIT8321 Forensics and Ethical Hacking, CA Lab Exercise 4
- System examined: VM securebank-lab (securebank-lab), SecureBank Cyber Fortress containers
- Lab source: github.com/frederickottache-sketch/secure-cyber-fortress (commit 03870ad)
- Time zone used: UTC

## Scope
Evidence collection and preservation only, on the isolated securebank-lab VM
and its six Docker containers. No real, public or third-party system is touched.

## Log (UTC time | action | result)
2026-11-01T13:15:46Z | Case opened, case folder created | OK
2026-11-01T14:16:44Z | Fetched SecureBank login page with curl from inside the VM (S06) | HTTP 200, login form shown
2026-11-01T14:16:44Z | Login as jane with wrong password (attempt 1 of 3), curl POST /login | Login failed. Try again
2026-11-01T14:16:46Z | Login as jane with wrong password (attempt 2 of 3), curl POST /login | Login failed. Try again
2026-11-01T14:16:48Z | Login as jane with wrong password (attempt 3 of 3), curl POST /login | Login failed. Try again
2026-11-01T14:16:50Z | Login as jane with correct password from db_init.sql | Welcome jane! View Accounts
2026-11-01T14:16:56Z | Login as john (password from db_init.sql), session cookie kept | Welcome john! View Accounts
2026-11-01T14:16:58Z | Opened /accounts as john | HTTP 200, accounts page shown
2026-11-01T14:17:00Z | Signed out john by discarding session cookie (app has no logout route) | Session ended client-side
2026-11-01T14:17:10Z | Note: /accounts was requested twice at 14:16:58Z (status check + page display), so two ACCOUNT_VIEW lines for john are expected | Clarification
2026-11-01T14:17:10Z | Started lecturer Phase 1 script: docker exec kali-attacker bash /scripts/attack.sh (targets securebank-app only) | Running
2026-11-01T14:17:28Z | Finished lecturer Phase 1 script (exit code 0) | See securebank.log and evidence/attack_timeline.txt
2026-11-01T14:18:00Z | Checked app log with tail (S07): LOGIN_FAILED x3 then LOGIN_SUCCESS for jane, john LOGIN_SUCCESS + ACCOUNT_VIEW, attack events from 172.18.0.7
| OK
2026-11-01T14:18:16Z | Saved collect_evidence.sh from GUIDE.md Step 4 unchanged (SHA-256 5d3ccf3b...1e03) | OK
2026-11-01T14:20:37Z | Ran collect_evidence.sh (S08): 24 items collected, hashed, registered, sealed read-only and archived | OK
2026-11-01T14:21:42Z | Reviewed case folder, evidence register, hashes.txt and chain of custody (S09a-S09d) | 24 items, all evidence files read-only
2026-11-01T14:21:56Z | Hashes verified, write-protection and tamper test done | All OK
2026-11-01T14:21:56Z | Note: tamper test needed chmod u+w on the /tmp working copy only (cp keeps the read-only mode); original evidence untouched, hash
81f65a71...d34cc unchanged (S10) | Deviation recorded
2026-11-01T14:38:30Z | Case package transferred to Mac and re-verified (S11): package SHA-256 f43cef12...195ecf matched in VM and on Mac | All OK
```